

Supplementary Material for "STARS: A Sampling and Threshold Sharing Solution for Network K-function Analytics"

Hongwei Ye
College of Computer Science and
Software Engineering,
Shenzhen University
Shenzhen, China
yehongwei2023@email.szu.edu.cn

Tsz Nam Chan
College of Computer Science and
Software Engineering,
Shenzhen University
Shenzhen, China
edisonchan@szu.edu.cn

Leong Hou U
Department of Computer and
Information Science,
University of Macau
Macao, China
ryanlhu@um.edu.mo

Dingming Wu
College of Computer Science and
Software Engineering,
Shenzhen University
Shenzhen, China
dingming@szu.edu.cn

Wei Tu
Ruisheng Wang
School of Architecture and Urban
Planning, Shenzhen University
Shenzhen, China
{tuwei,ruiswang}@szu.edu.cn

Joshua Zhexue Huang
College of Computer Science and
Software Engineering,
Shenzhen University
Shenzhen, China
zx.huang@szu.edu.cn

S1 Commonly Used Notations

In this section, we summarize the notations in Table I that are commonly used in this paper.

S2 Pseudocode of STARS

Algorithm I shows the pseudocode of STARS for computing an approximate network K -function plot, which has also been discussed in Section 3.2. As a remark, the implementation of STARS can be found in the Github link <https://github.com/fastnsa/STARS>.

S3 Further Discussion of NS and STARS

In this section, we provide further discussion of NS and STARS that has not been included in the main content of the paper. Here, we first point out three questions about NS and STARS. Then, we mention the implication of the answers to these questions.

Why does it not work for simply combining "sampling" with NS? In order to evaluate $A_{P,Q}(\tau)$ (see Equation 11), a straightforward approach is to adopt NS (see Section 2.2) for computing $C_{Q(e)}^{(p_i)}(\tau)$, i.e., maintaining four sets of data points on the edge $e = (u, v)$ with $Q(e)$ as the set of data points on this edge (see Figure I).

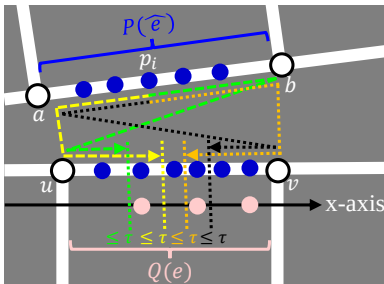


Figure I: Using the NS method for computing $C_{Q(e)}^{(p_i)}(\tau)$ (with $e = (u, v)$) for each data point p_i of the original dataset P on $\widehat{e} = (a, b)$ (i.e., $P(\widehat{e})$).

Algorithm I STARS for an approximate network K -function plot

Input: $G = (V, E)$, $R_1, R_2, \dots, R_L = P$, $\tau_1, \tau_2, \dots, \tau_D, \epsilon$
Output: $A_{R_\lambda, Q_\lambda}(\tau_\rho)$, where $1 \leq \lambda \leq L$ and $1 \leq \rho \leq D$

- 1: // Initialize $A_{R_\lambda, Q_\lambda}(\tau_\rho)$, $1 \leq \lambda \leq L$ and $1 \leq \rho \leq D$, to be 0
- 2: // Sampling
- 3: **for** each $e \in E$ **do**
- 4: **for** $1 \leq \lambda \leq L$ **do**
- 5: Construct $Q_\lambda(e)$ {Set $\delta = \frac{(n-1)\epsilon}{2n}$ in the SS method}
- 6: **end for**
- 7: **end for**
- 8: **for** each $e = (u, v) \in E$ **do**
- 9: $SPD(u) \leftarrow SPD_{\tau_D}(u)$, where (based on Dijkstra's algorithm)
- 10: $SPD(v) \leftarrow SPD_{\tau_D}(v)$
- 11: **for** each $\widehat{e} = (a, b) \in E$ **do**
- 12: **for** $1 \leq \lambda \leq L$ **do**
- 13: // Threshold sharing
- 14: Obtain $C_{R_\lambda(\widehat{e})}^{Q_\lambda(e)}(\tau_\rho)$, where $1 \leq \rho \leq D$
- 15: $A_{R_\lambda, Q_\lambda}(\tau_\rho) \leftarrow A_{R_\lambda, Q_\lambda}(\tau_\rho) + \frac{|R_\lambda(e)|}{|Q_\lambda(e)|} C_{R_\lambda(\widehat{e})}^{Q_\lambda(e)}(\tau_\rho)$ {Eq. 16}
- 16: **end for**
- 17: **end for**
- 18: Clear $SPD(u)$ and $SPD(v)$
- 19: **end for**
- 20: $A_{R_\lambda, Q_\lambda}(\tau_\rho) \leftarrow \frac{1}{n(n-1)} A_{R_\lambda, Q_\lambda}(\tau_\rho)$ {Eq. 16}
- 21: **return** $A_{R_\lambda, Q_\lambda}(\tau_\rho)$, where $1 \leq \lambda \leq D$ and $1 \leq \rho \leq D$

However, since using NS for computing $C_{Q(e)}^{(p_i)}(\tau)$ with all data points p_i in $P(\widehat{e})$ takes $O(|P(\widehat{e})| + |Q(e)|)$ time (see Section 2.2), the time complexity of NS for handling all edge pairs $(\widehat{e}, e)$ is

$$\sum_{\widehat{e} \in E} \sum_{e \in E} O(|P(\widehat{e})| + |Q(e)|) = O\left(n|E| + \frac{|E|^2}{\epsilon}\right) \quad (I)$$

Table I: Commonly used notations in this paper.

Notation	Description
<i>Notations about datasets</i>	
$G = (V, E)$	A road network with the node set V and the edge set E
$P = \{p_1, p_2, \dots, p_n\}$	A location dataset
n	Number of data points in the dataset P
$R_1, R_2, \dots, R_{L-1}$	$L - 1$ randomly generated datasets (with the same size n)
L	Total number of datasets
Q	A sampled dataset (obtained by the SS method for P)
<i>Notations about shortest path computation</i>	
$d_G(p_i, p_j)$	Shortest path distance between points p_i and p_j in G
T_{SP}	Time complexity of the shortest path algorithm
S_{SP}	Space complexity of the shortest path algorithm
<i>Notations about a network K-function plot</i>	
τ	Distance threshold of a network K -function
$\tau_1, \tau_2, \dots, \tau_D$	D distance thresholds
D	Number of distance thresholds
$K_P(\tau)$	The network K -function value at the threshold τ of the dataset P
$\mathbb{L}(\tau_\rho) (\mathbb{U}(\tau_\rho))$	the minimum (maximum) network K -function value at the threshold τ_ρ of $L-1$ random datasets
<i>Notations about NS and STARS</i>	
$C_{P(e)}^{(p_i)}(\tau)$	$(p_i, P(e))$ -count function
$\mathbb{C}_{Q(e)}^{P(\hat{e})}(\tau)$	Sum of $(p_i, Q(\hat{e}))$ -count function value for each p_i in $P(\hat{e})$
$\mathbb{C}_{P(\hat{e})}^{Q(e)}(\tau)$	Sum of $(p_k, P(\hat{e}))$ -count function value for each p_k in $Q(e)$
$A_{P,Q}(\tau)$	An approximate value for the network K -function $K_P(\tau)$
ϵ	A theoretically maximum absolute error between $A_{P,Q}(\tau)$ and $K_P(\tau)$
δ	A theoretically maximum absolute error between $KDE_X(q)$ and $KDE_S(q)$, where S is the subset of X (with one-dimensional data points) that is obtained by the SS method.

On the other hand, using NS to compute $C_{P(e)}^{(p_i)}(\tau)$ with all data points p_i in $P(\hat{e})$ also takes $O(|P(\hat{e})| + |P(e)|)$ time, which results in the time complexity of

$$\sum_{\hat{e} \in E} \sum_{e \in E} O(|P(\hat{e})| + |P(e)|) = O(n|E|) \quad (\text{II})$$

for handling all edge pairs $(\hat{e}, e)$. Therefore, this straightforward approach is unable to reduce the time complexity for generating a network K -function plot.

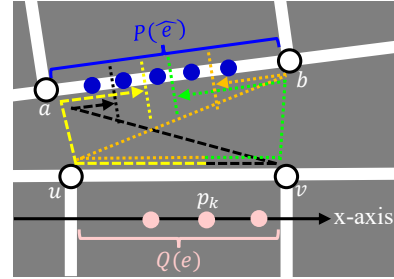
Why is it necessary to consider "evaluation order reversal" in the threshold sharing component of STARS? Suppose that we do not reverse the evaluation order, i.e., we directly compute $\mathbb{C}_{Q(e)}^{P(\hat{e})}(\tau)$

(see Equation 13) rather than $\mathbb{C}_{P(\hat{e})}^{Q(e)}(\tau)$ (see Equation 14). We establish those sorted intervals on the edge $e = (u, v)$ (rather than $\hat{e} = (a, b)$ in Figure 8 to Figure 10). Therefore, the time complexity of establishing sorted threshold positions and computing $\mathbb{C}_{Q(e)}^{P(\hat{e})}(\tau)$ becomes $O(D|P(\hat{e})| \log |P(\hat{e})| + |Q(e)|)$ (by swapping $Q(e)$ and $P(\hat{e})$ in Lemma 6 and Lemma 7), which results in the time complexity of $O(|E|T_{SP} + LnD|E| \log n + \frac{L|E|^2}{\epsilon})$ for generating an approximate network K -function plot (following the similar analysis in the paper). Note that this time complexity can be even larger than the one of NS (see Table 1). The main reason is that we maintain excessive (i.e., $D|P(\hat{e})|$) intervals on the edge $\hat{e} = (a, b)$ if we do not adopt the idea of "evaluation order reversal".

Can "sampling" and "evaluation order reversal" be combined with NS to further improve the efficiency of it? The answer to this question is no. Observe from Figure II (the reverse order of Figure 7) that the time complexity of computing $\mathbb{C}_{P(\hat{e})}^{(p_k)}(\tau)$ with all data points p_k in $Q(e)$ is $O(|Q(e)| + |P(\hat{e})|)$. Therefore, the time complexity of handling all edge pairs $(\hat{e}, e)$ is

$$\sum_{\hat{e} \in E} \sum_{e \in E} O(|Q(e)| + |P(\hat{e})|) = O(n|E| + \frac{|E|^2}{\epsilon})$$

which is the same as the one in Equation I (without using the idea of "evaluation order reversal"). Hence, this approach is still unable to reduce the time complexity of generating an approximate network K -function plot.

**Figure II: Combining the "sampling" and "evaluation order reversal" with the NS method.**

Implication. Based on the answer to the first question, we note that simply combining NS with the sampling method cannot reduce the time complexity for generating an approximate network K -function plot. Based on the answer to the second question, we note that the idea of "evaluation order reversal" is necessary in the threshold sharing component. Based on the answer to the third question, we also note that it is necessary to adopt the ideas of "establishment of sorted threshold positions" and "sweep line processing" (rather than simply adopting the NS method). Hence, we can conclude that all the ideas, "evaluation order reversal", "establishment of sorted threshold positions", and "sweep line processing", in the threshold sharing component are necessary to be considered for reducing the time complexity of generating an approximate network K -function plot.

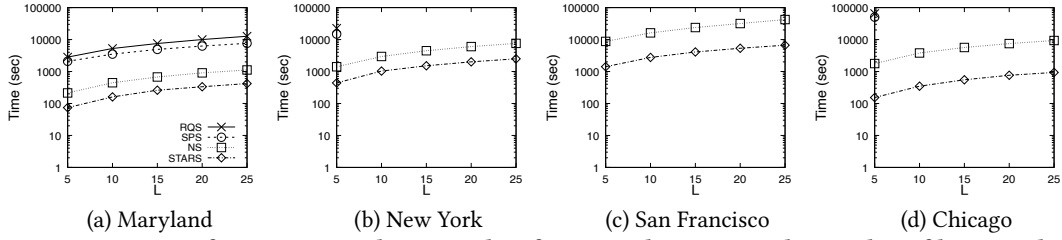


Figure III: Response time for computing the network K -function plot, varying the number of location datasets L .

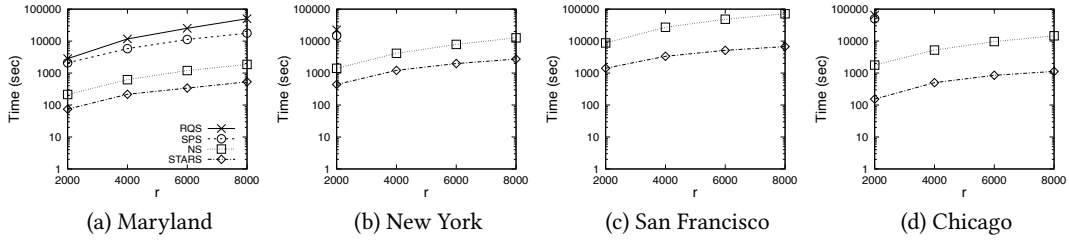


Figure IV: Response time for computing the network K -function plot, varying the maximum range of thresholds r .

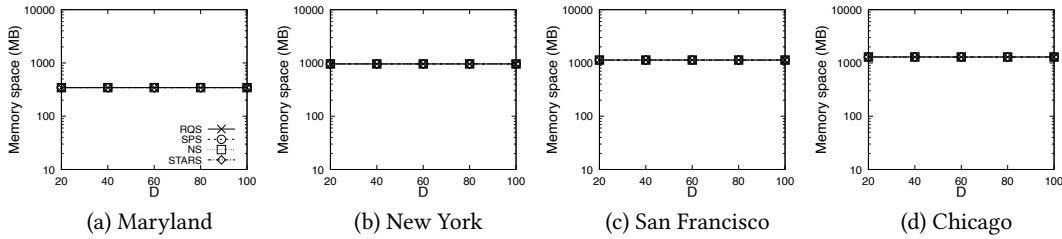


Figure V: Memory space consumption for computing the network K -function plot, varying the number of thresholds D .

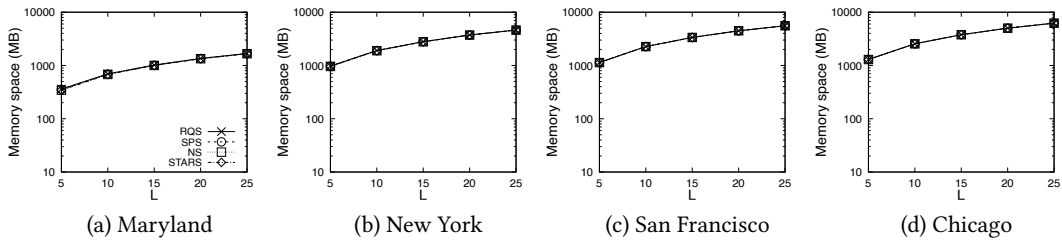


Figure VI: Memory space consumption for computing the network K -function plot, varying the number of location datasets L .

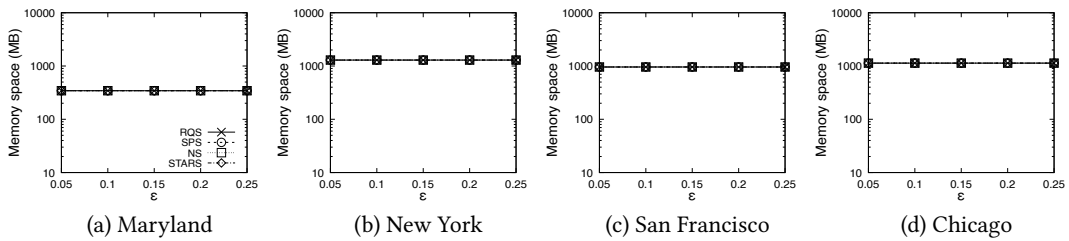


Figure VII: Memory space consumption for generating a network K -function plot, varying the error parameter ϵ .

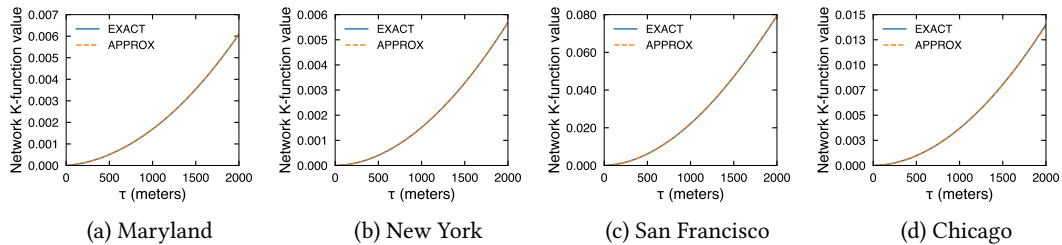


Figure VIII: Generating a network K -function plot with one of the randomly generated datasets using the exact and approximation methods, varying the threshold τ .

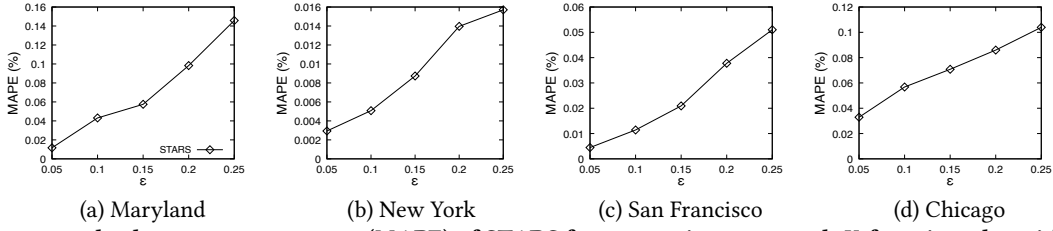


Figure IX: The mean absolute percentage error (MAPE) of STARS for generating a network K -function plot with the original dataset, varying the error parameter ϵ .

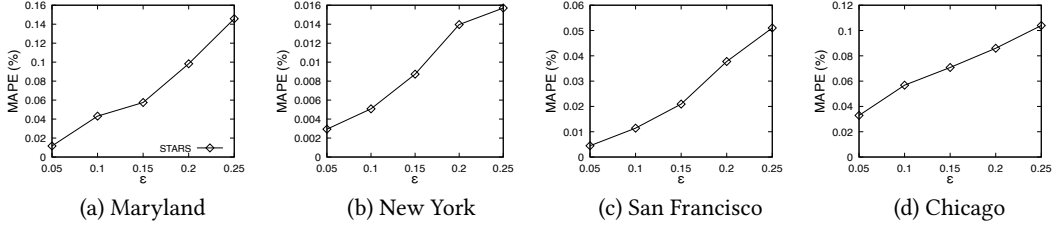


Figure X: The mean absolute percentage error (MAPE) of STARS for generating a network K -function plot with the one of the random datasets, varying the error parameter ϵ .

S4 Additional Time Efficiency Experiments

In this section, we further include two additional experiments for testing the time efficiency of generating a network K -function plot, which follows the same settings as Section 4.1.

Varying the number of datasets L . We investigate how the number of datasets L affects the response time of each method by choosing five L values (i.e., 5, 10, 15, 20, and 25). Since the time complexity of each method depends on L , the response time of each method increases with respect to this parameter (see Figure III). Observe that STARS can achieve 2.59x to 11.57x speedups compared with NS.

Varying the maximum range of thresholds r . Here, we test how the range $[0, r]$ (meters) affects the response time of each method, by choosing four r values, namely 2000, 4000, 6000, and 8000. Observe from Figure IV that the larger the r value, the larger the response time of each method. The main reason is that every method needs to process more data points if the threshold is larger (see Equation 17). With the lower time complexity of STARS, it can achieve 2.86x to 12.99x speedups compared with NS.

S5 Space Efficiency Experiments

In this section, we investigate the space efficiency (in terms of MB) of all methods for generating a network K -function plot.

Varying the number of thresholds D . To conduct this experiment, we first choose five D values, which are 20, 40, 60, 80, and 100, and then measure the memory space of all methods. Since STARS needs to maintain the additional sampled dataset on each edge, this method consumes higher memory space theoretically (see Table 1) compared with other methods. However, as this additional space consumption of STARS is not huge, this method has the similar memory space consumption compared with other methods (see Figure V) no matter which D we choose.

Varying the number of datasets L . We proceed to test how the number of datasets L affects the memory space consumption of each method, by choosing five L values, which are 5, 10, 15, 20, and

25. Since the term Ln in the space complexities of all methods (see Table 1) is the dominant term, we can observe from Figure VI that the space consumptions of all methods are (1) proportional to this parameter L and (2) similar to each other.

Varying the error parameter ϵ . Here, we further investigate how the absolute error parameter ϵ affects the memory space consumption of each method by choosing five values, including 0.05, 0.1, 0.15, 0.2, and 0.25. Figure VII shows the results of all methods. Observe that the memory space consumptions of all methods are similar to each other. The main reason is that the space complexities are mainly dominated by the term Ln (see Table 1). Hence, we conclude that STARS does not incur significant space overhead even though we choose small ϵ (e.g., 0.05).

S6 Additional Accuracy Experiments

In this section, we include additional subjective accuracy and objective accuracy experiments that have not been included in the main content.

Subjective accuracy. Recall from Figure 13 that using the exact (e.g., RQS, SPS, and NS) and approximation (STARS) methods can produce nearly the same curve with each original dataset P . Here, we also conduct this experiment for one of the randomly generated datasets (see Problem 1). With the non-trivial error guarantee (see Theorem 1), the curves that are generated from the exact and approximation methods also overlap with each other (see Figure VIII).

Objective accuracy. Here, we use another commonly used metrics, called Mean Absolute Percentage Error (MAPE) (see Equation III), for comparing the accuracy between the approximation method, STARS, and the exact methods.

$$MAPE = \frac{100}{D} \sum_{\rho=1}^D \frac{|A_{P,Q}(\tau_\rho) - K_P(\tau_\rho)|}{K_P(\tau_\rho)} \quad (III)$$

To conduct this experiment, we adopt the default settings in Section 4.1. Figure IX and Figure X show the mean absolute percentage error (MAPE) (see Equation III) of using STARS to generate the

network K -function plot with the original dataset and one of the random datasets, respectively, based on different error parameters ϵ . With the non-trivial error guarantee of STARS (see Theorem 1),

our method only incurs at most 0.1625% of MAPE for generating this plot.